

Designing Observable Services in Microservice Architecture

Hüsamettin Işıktas
Dept. of Computer Engineering
Yıldız Technical University
Istanbul, Türkiye
husamettin.isiktas@std.yildiz.edu.tr

Abstract—Deploying applications into production, particularly within a microservice architecture, necessitates a robust strategy for monitoring and troubleshooting. Unlike monolithic systems, microservices introduce distributed complexities where a single request may traverse multiple service instances, complicating failure detection and performance bottleneck analysis. This paper examines essential design patterns required to ensure service observability. It delineates the separation of responsibilities between development and operations and analyzes six fundamental patterns: Health Check API, Log Aggregation, Distributed Tracing, Exception Tracking, Application Metrics, and Audit Logging. The implementation of these patterns facilitates the tracking of runtime behavior, enables proactive alerting, and significantly reduces the mean time to resolution (MTTR) for production issues.

Index Terms—microservices, observability, distributed tracing, log aggregation, health check, devops

I. INTRODUCTION

Understanding runtime behavior is a critical requirement for any application deployed in a production environment. Stakeholders require visibility into key performance indicators such as request throughput, resource utilization, and the health status of service instances. Furthermore, proactive alerting mechanisms are essential to identify potential failures before they impact the end-user experience.

In a microservice architecture, the responsibility for system stability is shared. While hardware availability and infrastructure monitoring typically fall under the domain of operations, the internal manageability of services is the responsibility of the developers. To achieve this, specific observability patterns must be implemented. As illustrated in Fig. 1, these patterns expose the internal state and behavior of a service, enabling monitoring systems to visualize its health and performance.

The following patterns are fundamental for designing observable services:

- Health Check API
- Log Aggregation
- Distributed Tracing
- Exception Tracking
- Application Metrics
- Audit Logging

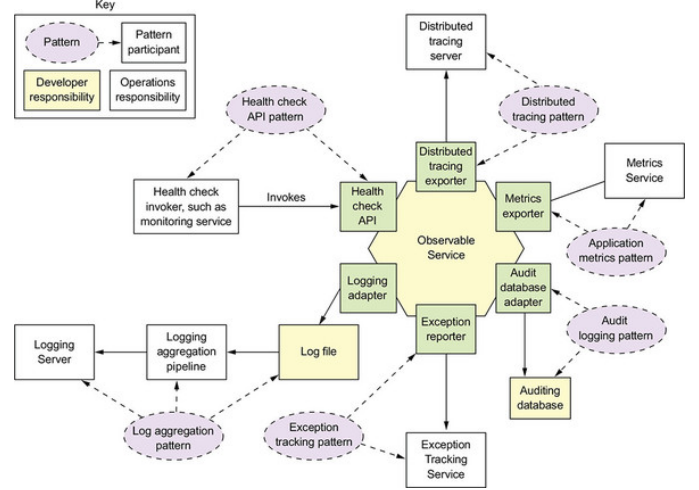


Fig. 1. The observability patterns enable developers and operations to understand the behavior of an application and troubleshoot problems. Developers are responsible for ensuring that their services are observable.

II. OBSERVABILITY PATTERNS

A. Health Check API

A service instance may be in a running state but unable to process requests effectively. This scenario can occur during the initialization phase or due to external dependency failures, such as the loss of a database connection. Routing traffic to such instances results in failed requests and degraded system reliability.

To mitigate this, services should implement the *Health Check API* pattern. This involves exposing a specific endpoint, such as `GET /health`, which reports the internal status of the service. As depicted in Fig. 2, the deployment infrastructure (e.g., Kubernetes or a Service Registry) periodically invokes this endpoint.

A standard HTTP 200 status code indicates that the service is healthy and ready to accept traffic. Conversely, a failure status (e.g., 503) signals the infrastructure to stop routing requests to that specific instance or to restart it.

B. Log Aggregation

In distributed systems, logs are generated by numerous service instances scattered across different servers or containers.

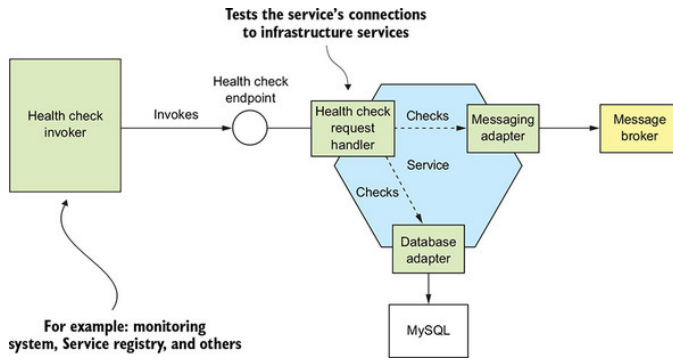


Fig. 2. A service implements a health check endpoint, which is periodically invoked by the deployment infrastructure to determine the health of the service instance.

Troubleshooting a failure by manually accessing individual log files is inefficient and often impractical.

The *Log Aggregation* pattern addresses this challenge by centralizing log management. As shown in Fig. 3, a log aggregation pipeline collects logs from all service instances and ships them to a centralized server. This centralized repository provides capabilities for searching, analyzing, and visualizing log data.

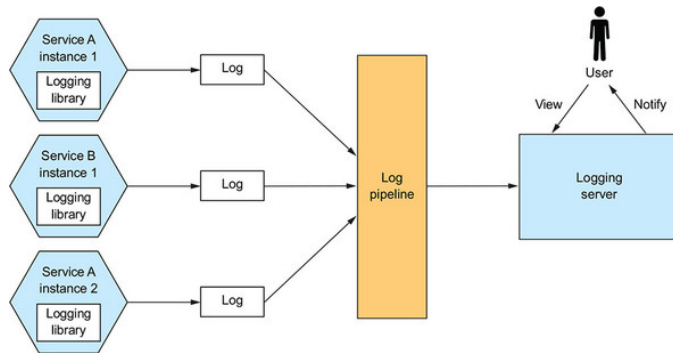


Fig. 3. The log aggregation infrastructure ships the logs of each service instance to a centralized logging server. Users can view and search the logs. They can also set up alerts.

Common implementations utilize the ELK stack (Elasticsearch, Logstash, Kibana) or similar cloud-native solutions, which also support alerting based on specific log patterns or error rates.

C. Distributed Tracing

Diagnosing latency issues in a microservice architecture is complex, as a single external request often triggers a chain of internal service calls. Without visibility into this chain, identifying which specific service is causing a bottleneck is difficult.

Distributed Tracing resolves this by assigning a unique identifier (Trace ID) to each external request. This ID is propagated through all subsequent calls within the system. As illustrated in Fig. 4, a distributed tracing server (such as Zipkin or Jaeger) aggregates this data to visualize the request flow and the time spent in each service.

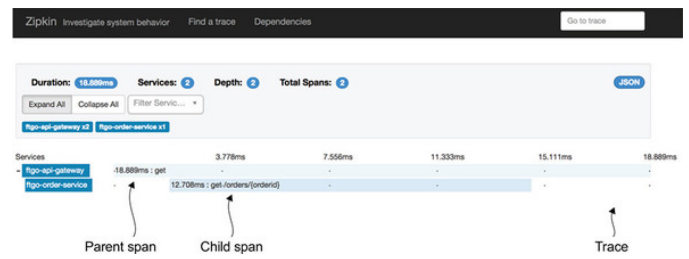


Fig. 4. The Zipkin server shows how the FTGO application handles a request that's routed by the API gateway to Order Service. Each request is represented by a trace consisting of spans.

Technically, this is implemented using an instrumentation library within the service code. As demonstrated in Fig. 5, the library manages the lifecycle of traces and spans, propagating context via HTTP headers (e.g., B3 propagation) and asynchronously reporting data to the tracing server.

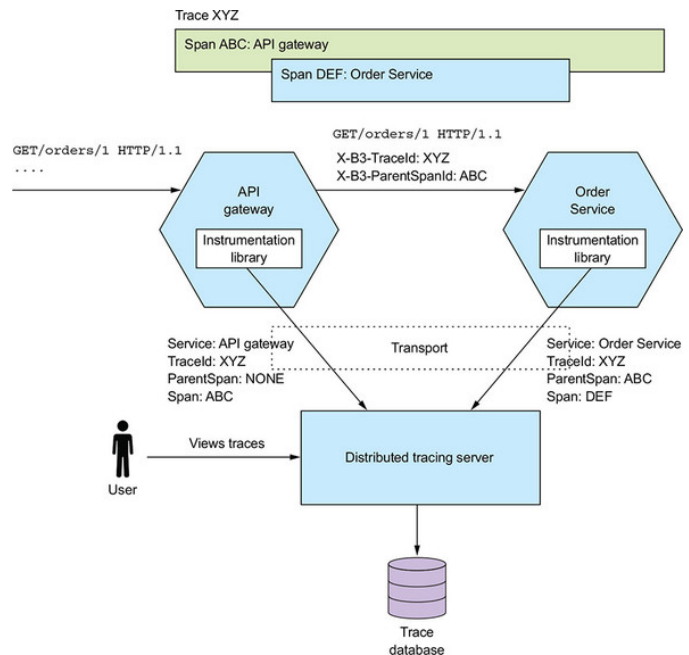


Fig. 5. Each service uses an instrumentation library. The library assigns an ID to each external request, propagates tracing state, and reports spans to the distributed tracing server.

D. Application Metrics

A comprehensive monitoring strategy requires collecting metrics from all layers of the technology stack. While infrastructure metrics (CPU, memory) are vital, they do not provide insight into business-level performance.

Application Metrics focus on service-specific indicators such as operation counters, gauges, and timers. Fig. 6 shows the architecture where metrics are collected and stored in a time-series database.

Data collection typically follows one of two models: the *pull model* (e.g., Prometheus scraping endpoints) or the *push model* (e.g., sending data to AWS CloudWatch). These metrics

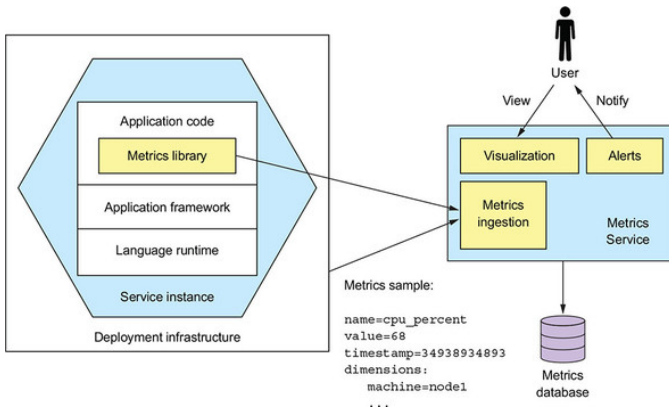


Fig. 6. Metrics at every level of the stack are collected and stored in a metrics service, which provides visualization and alerting.

enable the creation of dashboards and alerts for anomalies in business logic execution.

E. Exception Tracking

Traditional log-based exception handling suffers from duplication and a lack of issue tracking workflows. The *Exception Tracking* pattern proposes a centralized service dedicated to error management.

As shown in Fig. 7, services report exceptions directly to a tracking system (e.g., Sentry or Honeybadger). This system de-duplicates errors, groups them by root cause, and notifies developers. This approach streamlines the debugging process by providing context and stack traces in a unified interface, rather than requiring developers to grep through massive log files.

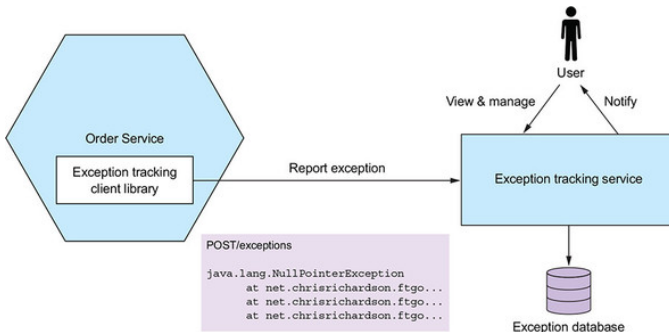


Fig. 7. A service reports exceptions to an exception tracking service, which de-duplicates exceptions and alerts developers. It has a UI for viewing and managing exceptions.

F. Audit Logging

Audit logging is essential for compliance, security, and customer support. It involves recording the specific actions taken by users, including the user identity, the action performed, and the affected business object.

There are three primary approaches to implementing audit logging in microservices:

- 1) **Embedding in Business Logic:** Developers explicitly write audit logs within the service methods. While straightforward, this intertwines logging code with business logic, reducing maintainability.
- 2) **Aspect-Oriented Programming (AOP):** This method uses AOP frameworks (e.g., Spring AOP) to intercept method calls and automatically generate audit logs. This separates concerns but may lack access to deeper business context inside the method.
- 3) **Event Sourcing:** In this architecture, state changes are stored as a sequence of events. This inherently provides a reliable audit trail for create and update operations, though recording read operations (queries) may still require additional mechanisms.

III. CONCLUSION

Observability is not merely an operational convenience but a fundamental requirement for the reliability of microservice architectures. By implementing the patterns discussed—including Health Checks, Log Aggregation, Distributed Tracing, Application Metrics, Exception Tracking, and Audit Logging—engineering teams can ensure system transparency. These practices enable the effective monitoring of system health, facilitate rapid root cause analysis, and ultimately minimize the impact of failures on the end-user.

ACKNOWLEDGMENT

This paper is based on the concepts of observability patterns in microservice architectures as discussed in [1].

REFERENCES

- [1] C. Richardson, *Microservices Patterns: With examples in Java*. Manning Publications, 2018, ch. 11.
- [2] Microservices.io, “Observability Patterns.” [Online]. Available: <http://microservices.io/patterns/observability>. [Accessed: Dec. 30, 2025].